IILM UNIVERSITY

GREATER NOIDA

SUMMER INTERNSHIP REPORT

PYTHON DEVELOPER INTERNSHIP

Submitted By	Anjana Kumari
Enrollment Number	2410031184
Programme	B.Tech – Computer Science & Engineering
Year	3rd Year
Host Organization	Codec Technologies Pvt. Ltd.
Internship Period	02 July 2026 – 01 August 2026

AICTE & ICAC Approved • 1 Month Internship Program

NOTE ON REPORT PREPARATION

This report has been expanded into a complete academic-style internship report using the information available in the internship certificate and the Python Developer Intern role. The certificate verifies the student, host organization, role, program, and internship dates. The detailed project narrative, workflow, sample implementation, technologies, and results in the following chapters are a professionally structured report draft created to make the submission complete; they should be checked against the actual work performed during the internship before formal submission.

DECLARATION

I, Anjana Kumari, Enrollment No. 2410031184, student of B.Tech Computer Science and Engineering, 3rd Year at IILM University, Greater Noida, hereby submit this Summer Internship Report for the internship completed at Codec Technologies Pvt. Ltd. as a Python Developer Intern.

The internship period covered 02 July 2026 to 01 August 2026. This report presents the internship experience, technical learning, project-oriented work, development process, and professional skills gained during the internship.

Signature of Student: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to IILM University, Greater Noida, for providing me with the opportunity to undertake a Summer Internship as part of my B.Tech Computer Science and Engineering curriculum.

I am thankful to Codec Technologies Pvt. Ltd. for providing the internship opportunity and industry exposure in the role of Python Developer Intern. The internship helped me understand how programming concepts are applied to practical software-development activities.

I would also like to acknowledge the guidance, feedback, and learning environment associated with the internship program. The experience encouraged me to improve my programming discipline, debugging approach, documentation habits, and confidence in developing Python-based applications.

Finally, I thank my faculty members, mentors, friends, and family for their support and encouragement throughout the internship.

EXECUTIVE SUMMARY

This report documents the Summer Internship completed by Anjana Kumari, a third-year B.Tech Computer Science and Engineering student at IILM University, Greater Noida. The internship was carried out at Codec Technologies Pvt. Ltd. in the role of Python Developer Intern.

The internship certificate records that the program was a 1 Month AICTE & ICAC Approved internship conducted from 02/07/2026 to 01/08/2026. The central technical area of the internship was Python development.

The report presents the internship objectives, organization profile, development methodology, Python-oriented technical learning, a structured project narrative, database and user-interface concepts, testing, challenges, results, professional development, and future scope.

The report is designed to demonstrate the connection between academic knowledge and practical software development, with emphasis on clean programming, modular design, data management, validation, testing, and documentation.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background

The software industry requires developers to combine programming knowledge with structured problem solving, testing, documentation, and the ability to work with changing requirements. Academic courses provide the foundation, while internships provide an environment in which these concepts can be applied in a practical context.

1.2 Internship Context

The internship was undertaken as a Python Developer Intern at Codec Technologies Pvt. Ltd. for one month. Python was the primary development area identified in the certificate.

1.3 Importance of Python

Python is widely used for application development, automation, data processing, scripting, APIs, testing, and artificial intelligence. Its readable syntax and extensive standard library make it suitable for learning software development while also supporting practical applications.

1.4 Student Profile

Submitted By	Anjana Kumari
Enrollment Number	2410031184
Programme	B.Tech – Computer Science & Engineering
Year	3rd Year
Host Organization	Codec Technologies Pvt. Ltd.
Internship Period	02 July 2026 – 01 August 2026

2. ORGANIZATION PROFILE

2.1 Codec Technologies Pvt. Ltd.

Codec Technologies Pvt. Ltd. is the organization named on the internship certificate. The certificate records the internship as a one-month AICTE & ICAC Approved program and identifies the assigned role as Python Developer Intern.

2.2 Internship Certification

Organization	Codec Technologies Pvt. Ltd.
Role	Python Developer Intern
Program	1 Month AICTE & ICAC Approved Internship
Period	02/07/2026 – 01/08/2026
Program Manager	Dr. Anurag Shrivastava

2.3 Professional Environment

The internship environment provided an opportunity to view software development as a process rather than only as code writing. A professional workflow generally involves understanding requirements, breaking work into modules, implementing functionality, testing outputs, fixing defects, and documenting the final result.

3. INTERNSHIP OVERVIEW AND OBJECTIVES

3.1 Duration and Role

The internship was completed over one month, from 02 July 2026 to 01 August 2026. The role documented on the certificate is Python Developer Intern.

3.2 Objectives

- Strengthen Python programming fundamentals and object-oriented programming concepts.
- Understand modular software development and reusable code.
- Practice input validation, exception handling, and debugging.
- Work with structured data and persistent storage concepts.
- Develop a small Python application from requirements to testing.
- Improve technical documentation, presentation, and professional communication.

3.3 Expected Professional Outcomes

By the end of the internship, the intended outcome was to become more comfortable with converting a software requirement into a working Python solution, testing it systematically, identifying defects, and explaining the implementation clearly.

4. ROLE AND RESPONSIBILITIES

4.1 Role

The official role was Python Developer Intern. Within a structured Python development internship, the role can be represented through programming, testing, debugging, documentation, and application-development activities.

4.2 Responsibilities

- Understand assigned requirements and convert them into logical programming steps.
- Write readable and modular Python code.
- Use functions and classes to separate responsibilities.
- Implement input validation and exception handling.
- Store and retrieve application data using a lightweight database.
- Test individual functions and complete user flows.
- Debug errors and improve reliability.
- Document the application, workflow, and results.

4.3 Development Practices

The development approach emphasized small modules, meaningful variable and function names, separation of user-interface and data operations, validation before database operations, and testing after each major feature.

5. TECHNOLOGIES AND TOOLS

The following stack is used in the prepared project narrative to demonstrate a coherent Python development workflow. These items should be verified against the actual internship environment before final institutional submission.

Technology / Tool	Purpose	Application in Project
Python 3	Core programming language	Business logic, validation, application flow
Tkinter	GUI toolkit	Forms, buttons, tables, user interaction
SQLite	Relational database	Persistent student records
VS Code	Code editor / IDE	Writing and debugging Python code
Git / GitHub	Version control	Tracking code changes and maintaining versions
Python standard library	Supporting utilities	Validation, dates, file and exception handling

6. PROJECT OVERVIEW

6.1 Project Title

Python-Based Student Management System

6.2 Project Description

The project is a desktop-based student management application designed to maintain student information in an organized digital form. The application provides a simple interface for adding, viewing, searching, updating, and deleting student records.

6.3 Problem Statement

Managing student records manually can result in duplicate entries, difficulty locating information, inconsistent formatting, and increased effort when updating records. A lightweight computerized system can centralize records and make common operations faster and more consistent.

6.4 Proposed Solution

The proposed solution uses Python for application logic, Tkinter for the user interface, and SQLite for persistent storage. The system validates user input before storing data and provides separate operations for record management.

6.5 Key Features

- Add a new student record.
- Display stored student records.
- Search records using a student identifier or name.
- Update existing information.
- Delete a selected record after confirmation.
- Validate required fields and basic data formats.
- Persist records in a local SQLite database.
- Handle invalid input and database errors gracefully.

7. REQUIREMENT ANALYSIS

7.1 Functional Requirements

ID	Requirement	Description
FR-01	Add Record	User can enter and save a new student record.
FR-02	View Records	System displays stored records in a tabular form.
FR-03	Search	User can find records using ID/name.
FR-04	Update	Existing student details can be modified.
FR-05	Delete	Selected record can be removed after confirmation.
FR-06	Validation	System rejects missing or invalid input.

7.2 Non-Functional Requirements

- Usability: interface should be simple for a first-time user.
- Reliability: invalid input should not terminate the application.
- Maintainability: code should be divided into logical functions/modules.
- Performance: common CRUD operations should complete quickly for a small local dataset.
- Data integrity: student IDs should be treated as unique identifiers.

8. SYSTEM DESIGN AND ARCHITECTURE

8.1 High-Level Architecture

The system follows a simple three-layer conceptual structure:

1. Presentation Layer – Tkinter forms, buttons, labels, and record table.
2. Application Layer – validation, business rules, search, update, delete, and control flow.
3. Data Layer – SQLite database used for persistent student records.

8.2 Data Flow

User → GUI Form → Input Validation → Python Application Logic → SQLite Database → Result / Status Message → GUI

8.3 Module Structure

Module	Main Responsibility	Examples
main.py	Application startup	Create window, initialize database
ui.py	User interface	Forms, buttons, table display
database.py	Database operations	INSERT, SELECT, UPDATE, DELETE
validation.py	Input checks	Required fields, formats, uniqueness
utils.py	Helper operations	Messages, formatting, reusable utilities

9. IMPLEMENTATION

9.1 Database Initialization

At application startup, the program creates or connects to the SQLite database and ensures that the required student table exists. This prevents the application from failing when it is launched for the first time.

9.2 Add Student

The Add Student workflow collects the student's identifier, name, course, year, email, and contact details. Validation is performed before the INSERT operation. Duplicate identifiers are rejected to preserve record uniqueness.

9.3 View and Search

The View operation retrieves records using a SELECT query and displays them in a table. Search applies a condition to the identifier or name so that the user can locate a specific record without manually scanning the entire list.

9.4 Update and Delete

Update first identifies the selected record and then writes the changed values back to the database. Delete removes the selected record only after a confirmation step. These controls reduce accidental modification of data.

9.5 Exception Handling

Database and input operations are protected with exception handling so that errors can be converted into understandable status messages rather than causing an abrupt application termination.

10. DATABASE DESIGN

10.1 Student Table

Field	Type	Key	Required	Purpose
student_id	INTEGER	PRIMARY KEY	Yes	Unique student identifier
name	TEXT	—	Yes	Student full name
course	TEXT	—	Yes	Academic course
year	INTEGER	—	Yes	Current academic year
email	TEXT	—	No	Contact email
phone	TEXT	—	No	Contact number

10.2 CRUD Operations

- Create – insert a new student record.
- Read – retrieve all or filtered student records.
- Update – modify selected record information.
- Delete – remove a selected record.

11. TESTING AND VALIDATION

11.1 Testing Strategy

Testing was organized around individual functions and complete user workflows. Normal inputs, missing fields, duplicate identifiers, invalid values, search queries, update operations, and delete confirmations were considered.

Test ID	Scenario	Expected Result	Status
T01	Add valid record	Record saved successfully	Pass
T02	Missing required field	Validation message displayed	Pass
T03	Duplicate student ID	Duplicate rejected	Pass
T04	Search existing record	Matching record displayed	Pass
T05	Search unknown value	No-match message/display	Pass
T06	Update record	Updated values displayed	Pass
T07	Delete with confirmation	Record removed	Pass
T08	Invalid numeric input	Input rejected safely	Pass

11.2 Validation Principles

- Required-field validation prevents incomplete records.
- Type validation prevents incompatible values.
- Uniqueness checks protect primary-key integrity.
- Exception handling prevents unexpected application crashes.
- Confirmation dialogs reduce accidental deletion.

12. CHALLENGES AND SOLUTIONS

Challenge	Solution / Approach	Learning
Input validation	Added checks before database operations.	Validation is essential for reliable software.
Duplicate records	Used unique student identifier.	Data integrity must be designed into the system.
Database errors	Used exception handling and clear messages.	Errors should be handled gracefully.
UI and logic coupling	Separated UI and database responsibilities.	Modular design improves maintainability.
Debugging	Tested individual operations and complete flows.	Small, repeatable tests speed up debugging.

13. LEARNING OUTCOMES

13.1 Technical Learning

- Improved understanding of Python syntax and program structure.
- Applied functions, classes, conditional logic, loops, and exception handling.
- Understood CRUD operations and basic relational database concepts.
- Practiced GUI development using a Python toolkit.
- Learned the importance of validation and testing.
- Improved debugging and code organization skills.

13.2 Professional Learning

- Breaking a larger requirement into smaller tasks.
- Maintaining readable and documented code.
- Testing before considering a feature complete.
- Communicating technical ideas in a structured way.
- Managing time across implementation, debugging, and documentation.

13.3 Academic Relevance

The internship connects programming, database management, software engineering, object-oriented programming, and problem-solving concepts studied in the B.Tech CSE curriculum with a practical application-development workflow.

14. RESULTS AND DISCUSSION

14.1 Project Result

The prepared project demonstrates a complete basic record-management workflow: a user can enter student information, validate it, store it in a database, retrieve records, search them, update information, and delete selected records.

14.2 Benefits

- Reduces repetitive manual record handling.
- Provides centralized local storage.
- Makes searching and updating records easier.
- Improves consistency through validation.
- Demonstrates a practical end-to-end Python application workflow.

14.3 Limitations

- Designed primarily for a small local dataset.
- SQLite is local rather than a multi-user cloud database.
- Authentication and role-based access are not included.
- No remote backup or deployment layer is included in the basic version.

15. FUTURE SCOPE

- Add login and role-based access for administrators and staff.
- Migrate from SQLite to MySQL/PostgreSQL for larger multi-user deployments.
- Build a web version using Flask or Django.
- Add CSV/Excel import and export.
- Add automated database backup and restore.
- Add advanced search, filtering, sorting, and reporting.
- Deploy the application to a managed server or cloud environment.
- Add automated unit and integration tests.
- Introduce an API layer for integration with other academic systems.

16. CONCLUSION

The Summer Internship at Codec Technologies Pvt. Ltd. was an important opportunity to gain industry-oriented exposure in the role of Python Developer Intern. The one-month program, conducted from 02 July 2026 to 01 August 2026, complemented the academic learning of B.Tech Computer Science and Engineering.

The project-oriented work presented in this report demonstrates how Python can be used to develop a structured application with a graphical interface, database storage, validation, CRUD operations, and error handling. The development process also highlights the importance of modular design, testing, debugging, and documentation.

Overall, the internship strengthened technical confidence and provided a foundation for further learning in Python development, databases, software engineering, web development, and application deployment.

17. REFERENCES

- Python Documentation – Python programming language reference and standard library documentation.
- SQLite Documentation – SQL and SQLite database concepts.
- Tkinter / Python GUI documentation – desktop interface development concepts.
- Git Documentation – version control concepts and workflows.
- Internship Certificate issued by Codec Technologies Pvt. Ltd.
- Academic coursework and laboratory exercises in B.Tech Computer Science and Engineering.

18. INTERNSHIP CERTIFICATE

The following certificate is the primary supporting document for the internship details used in this report. It identifies Anjana Kumari, Codec Technologies Pvt. Ltd., the Python Developer Intern role, the one-month AICTE & ICAC Approved program, and the internship period.

APPENDIX A – PROJECT MODULE SUMMARY

Module	Inputs	Processing	Output
Student Registration	ID, name, course, year, email, phone	Validate and insert	Success/error message
Record Display	None	SELECT records	Table of students
Search	ID or name	Filter query	Matching records
Update	Selected record + changes	Validate and UPDATE	Updated record
Delete	Selected record	Confirm + DELETE	Removal confirmation
Validation	User-entered values	Required/type/format checks	Validation result
Database	SQL operations	Execute and handle errors	Persistent data

APPENDIX B – SAMPLE PYTHON LOGIC

Illustrative pseudocode for the project workflow:

4. START APPLICATION
5. CONNECT TO SQLITE DATABASE
6. CREATE TABLE IF NOT EXISTS STUDENT
7. DISPLAY MAIN WINDOW
8. WAIT FOR USER ACTION
9. IF ADD: VALIDATE INPUT → INSERT RECORD → REFRESH VIEW
10. IF SEARCH: READ QUERY → SELECT MATCHING RECORDS → DISPLAY
11. IF UPDATE: VALIDATE CHANGES → UPDATE RECORD → REFRESH VIEW
12. IF DELETE: CONFIRM → DELETE RECORD → REFRESH VIEW
13. HANDLE ERRORS → SHOW USER-FRIENDLY MESSAGE
14. END

APPENDIX C – INTERNSHIP SUMMARY

Student	Anjana Kumari
Enrollment No.	2410031184
Programme	B.Tech CSE, 3rd Year
University	IILM University, Greater Noida
Organization	Codec Technologies Pvt. Ltd.
Role	Python Developer Intern
Duration	02 July 2026 – 01 August 2026

End of Report